

再帰下降パーサ① — 式を解析する

今日のゴール

`language_spec.md` の EBNF の式の階層を、そのまま関数の階層に写して、Core プロファイルの式パーサを自作する。86 本の式コーパスで、AST が scaffold の Parser と完全一致することを golden test で確認する。

1 この回の位置づけ

- フロントエンド発展シリーズの第 2 回（選択制）。前提は F1
- F1 で作ったトークン列を、今度は木にする。F2 は式、F3 で文、F4 で宣言・型を扱い、F4 の終わりで scaffold の Parser を完全に置き換える
- 編集するのは `myparser.py` だけ。Node と ND_* 定数は scaffold の `ast_def.py` から借りる。AST の形は既に決まっている。作るのは「組み立てる側」である
- `sizeof` は型のパースが必要なので F4 で扱う

2 再帰下降とは — 規則 = 関数

F0 の CYK 法は、どんな文法でも扱える代わりに $O(n^3)$ の表を作った。再帰下降構文解析は、表を作らない。

文法の 1 規則を 1 つの関数にする。関数は、自分の規則の形どおりにトークンを読み進め、対応する部分木を返す。

`language_spec.md` の式の EBNF は、優先順位の低い順に 10 段の規則が並んでいる。

```
add_expr ::= mul_expr { ('+' | '-') mul_expr }  
mul_expr ::= unary_expr { ('*' | '/' | '%') unary_expr }
```

この 2 行は、そのまま 2 つの関数 `parse_add` と `parse_mul` になる。「`add_expr` の中に `mul_expr` が出てくる」は「`parse_add` が `parse_mul` を呼ぶ」に対応する。



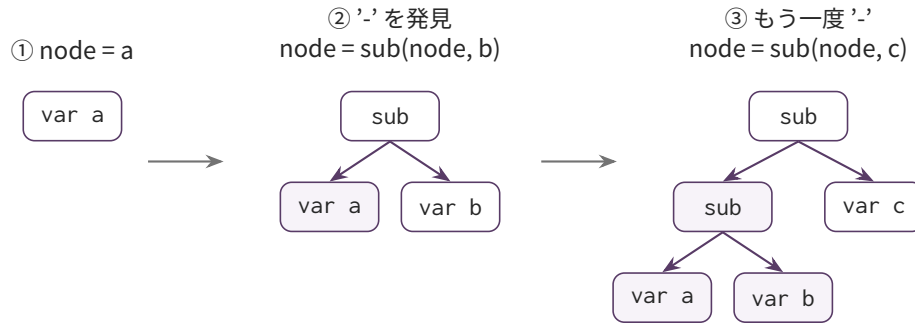
EBNF の 1 規則 = 1 関数。各関数は自分のレベルの演算子だけを処理し、それより優先順位の高い部分は 1 段下の関数に任せる

図 1 EBNF の優先順位階層 = 関数の呼び出し階層

各関数は**自分のレベルの演算子だけ**を処理し、それより優先順位の高い部分は 1 段下の関数に丸ごと任せる。この構造だけで、`1 + 2 * 3` の `*` が先に結ばれる。F0 で「優先順位は文法の書き方そのもの」と学んだが、その文法を関数に写した結果がこの階段である。

3 左結合はループで書く

`a - b - c` は `(a - b) - c` でなければならない (左結合)。これはループで書ける。



いままでの結果 (色つき) を lhs に入れるので、木は左へ左へ伸びる = 左結合

(a - b) - c と同じ意味になる。右結合にしたい '=' はループではなく、
parse_assign が自分自身を再帰することで右へ伸ばす

図2 左結合ループが a - b - c を組み立てる様子

```
def parse_add(self):
    """add_expr ::= mul_expr { ('+' | '-') mul_expr }"""
    node = self.parse_mul()
    while self.cur.sval in ('+', '-'):
        kind = ND_ADD if self.cur.sval == '+' else ND_SUB
        self.pos += 1
        node = Node(kind, lhs=node, rhs=self.parse_mul())
    return node
```

EBNF の { ... } (0 回以上の繰り返し) が while に、そのまま対応している。

ポイントは lhs=node である。いままでの結果を左の子に入れて、新しい親を作る。これを繰り返すと木は左へ左へ伸び、左結合になる。

4 2 回書いたら共通化する — parse_binary

parse_add と parse_mul を書き終わると、違いが「演算子の表」と「1 段下の関数」だけであることに気づくはずである。そこで共通部品にまとめる。

```
def parse_binary(self, op_map, next_fn):
    node = next_fn()
    while self.cur.sval in op_map:
        kind = op_map[self.cur.sval]
        self.pos += 1
```

```

        node = Node(kind, lhs=node, rhs=next_fn())
    return node

def parse_land(self):
    return self.parse_binary({'&&': ND_AND}, self.parse_bitor)

```

残りの二項レベルは、この 1 行ずつで書ける。

レベル	op_map
parse_lor	{' ': ND_OR}
parse_land	{'&&': ND_AND}
parse_bitor / parse_bitxor / parse_bitand	{' ': ND_BITOR} など
parse_eq	{'==': ND_EQ, '!=': ND_NE}
parse_shift	{'<<': ND_SHL, '>>': ND_SHR}

実は scaffold の `parser.py` もまったく同じ `_parse_binary` を持っている。黒箱の中身は、いま自分が書いたものと同じである。

5 例外が 1 つ — `parse_rel` の swap

比較のレベルだけは `parse_binary` で書けない。コマ 5 の資料に「`ND_GT` や `ND_GE` は AST に現れない」とあったのを覚えているだろうか。その正体がここにある。

```

if op == '>':
    node = Node(ND_LT, lhs=rhs, rhs=node)    # a > b は b < a として作る

```

`>` と `>=` は、左右の子を入れ替えて `Lt / Le` に正規化する。こうしておくと、コード生成側は `Lt` と `Le` の 2 種類だけ扱えばよくなる。「パーサで形を揃えて、後段を楽にする」という設計判断が AST に刻まれている例である。

6 単項と代入は「再帰」で書く

前置演算子は自分自身を再帰する。 `-- x` が `(neg (neg x))` になる。

```
def parse_unary(self):
    if self.consume_if('-'):
        return Node(ND_NEG, operand=self.parse_unary())
    ...
    return self.parse_postfix()
```

代入は右結合なので、ループではなく再帰で右へ伸ばす。`a = b = c` は `(assign a (assign b c))` である。

```
def parse_assign(self):
    node = self.parse_lor()
    if self.consume_if('='):
        return Node(ND_ASSIGN, lhs=node, rhs=self.parse_assign()) # 右結合
    return node
```

補足

仕様の EBNF は `unary_expr '=' assign_expr` なのに、なぜ `lor` まで読むのか。
`lor_expr` は `unary_expr` を含むので、「先に `lor` まで読んでしまい、`=` が来たらそれを代入の左辺とみなす」ことができる。`1 + 2 = x` のような不正な左辺はこの段階では通ってしまうが、左辺が `lvalue` かどうかの検査は意味解析 (`codegen_lval`) に任せる
 — EBNF の注釈「`lvalue` 制約は意味解析フェーズで検査」の実装がこれである。

7 後置はループで連なる — postfix

`p->next->val` や `a[i][j]` は、`primary` の結果に後置が左から積み重なる。

```
(member "->" "val" (member "->" "next" (var "p")))
```

後置がある限り回るループで、`[ expr ]` は `ND_INDEX`、`->` と `.` は `ND_MEMBER` (`is_arrow` で区別) を作る。いままでの結果を子に入れて新しい親を作る点は、左結合ループと同じ形である。

8 primary —再帰の輪が閉じる

最下段の `parse_primary` は、リテラル・変数・関数呼び出し・カッコを扱う。

現在のトークン	作るもの
TK_NUM / TK_CHAR	<code>Node(ND_NUM, val=...)</code>
TK_STR	<code>Node(ND_STR, sval=...)</code>
TK_IDENT のあとに (	<code>Node(ND_CALL, name=..., args=[...])</code>
TK_IDENT	<code>Node(ND_VAR, name=...)</code>
(	<code>parse_expr()</code> を読んで) を expect

(`expr`) で一番下の関数が一番上の関数を呼び戻す。図の再帰の輪がここで閉じ、`(a + b) * c` のような入れ子が自然に処理される。

補足

先読み 1 トークンで足りる — LL(1)。どの関数も、現在のトークン（と時々 `peek(1)`）を見るだけで進む道が決まる。このように「先読み 1 トークンで迷いなく解析できる」文法のクラスを LL(1) と呼ぶ。Core プロファイルの文法は、意図的にこの形に設計されている。唯一きわどいのは `sizeof(x)` の (の次が型か式かの判定で、`scaffold` は `peek(1)` と `typedef` 名の表で解決している (F4 で扱う)。

9 実装

`myparser.py` のスケルトンは `ExprParser` クラスになっている。トークン操作 (`cur / peek / advance / consume_if / expect / error`) は完成済み。各レベルの関数は最初は素通し (1 段下を呼ぶだけ) になっており、上から順に肉付けする。

Step	実装対象	内容
1	<code>parse_primary</code>	リテラル・変数・カッコ

Step	実装対象	内容
2	<code>parse_add / parse_mul</code>	左結合ループを手で 2 回書く
3	<code>parse_binary</code> + 残りのレベル + <code>parse_rel</code>	共通化と <code>swap</code> 正規化
4	<code>parse_unary / parse_postfix</code> + 関数呼び出し	前置の再帰・後置のループ
5	<code>parse_assign</code>	右結合の再帰

素通しのため、まだ実装していない演算子を含む式は「式の後にトークンが余っています」というエラーになる。これは「その演算子を誰も食べずに残した」という意味で、実装が進むと消えていく。

途中経過は目視でも確認できる。出力は `parse_viewer.py` と同じ S 式である。

```
python3 myparser.py '1 + 2 * 3'
```

10 テスト

10.1 単体テスト (Step ごと)

```
python3 check.py
```

期待値は講義で見慣れた S 式で書いてある。`(lt (var "b") (var "a"))` が `a > b` の正解である、という `swap` の確認も含む。

10.2 golden test (scaffold との突き合わせ)

```
python3 golden.py
```

全演算子・優先順位の組み合わせ・結合方向・`postfix` の連鎖を網羅した 86 本の式について、

scaffold の Parser と AST を構造比較する (`line` は比較しない)。全式 **PASS** がこの回の完了条件である。

11 発展課題

1. **sizeof** の先取り: scaffold の `_parse_sizeof` と `_peek_is_type` を読み、自作パーサに移植する (F4 の予習になる)
2. 三項演算子 `a ? b : c` を追加する (ヒント: 右結合。Core 外なので `golden` は対象外)
3. エラーメッセージの改善: 「`(`」が期待されました」に加えて、対応する開きカッコの行番号も表示する
4. 数を数える: 自作パーサの関数呼び出し回数を数え、式の長さに対して線形であることを確かめる (CYK の $O(n^3)$ との対比)

補足

コラム: 10 個の関数を 1 個にする方法。 二項演算のレベルを「演算子ごとの優先順位の数値」を引数に持つ 1 つの関数 `parse_expr(min_prec)` にまとめる書き方があり、precedence climbing あるいは Pratt parsing と呼ばれる。実務のパーサ (clang など) でも使われる技法だが、「文法の階層がそのままコードに見える」教育的な美しさは 10 関数方式にある。興味があれば `parse_binary` をさらに一般化してみるとよい。

12 次回予告

式が読めるようになった。F3 では文 (`if / while / for / return / ブロック / 式文`) に進む。文の解析は「先頭のキーワードで分岐するだけ」なので、実は式よりずっと素直である。コマ 5 で見た「`else` は最も内側の `if` に結合する」(dangling else) が再帰下降で自然に解決される様子も確かめる。